

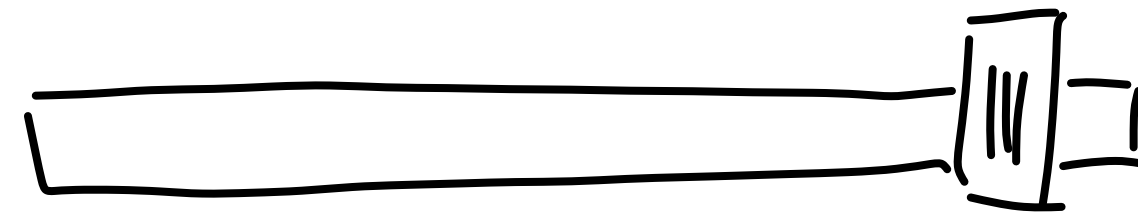
Zero-Knowledge Proofs for Trust, Privacy, and Scalability

Zero Knowledge in Blockchain

Blockchains



DECENTRALISATION

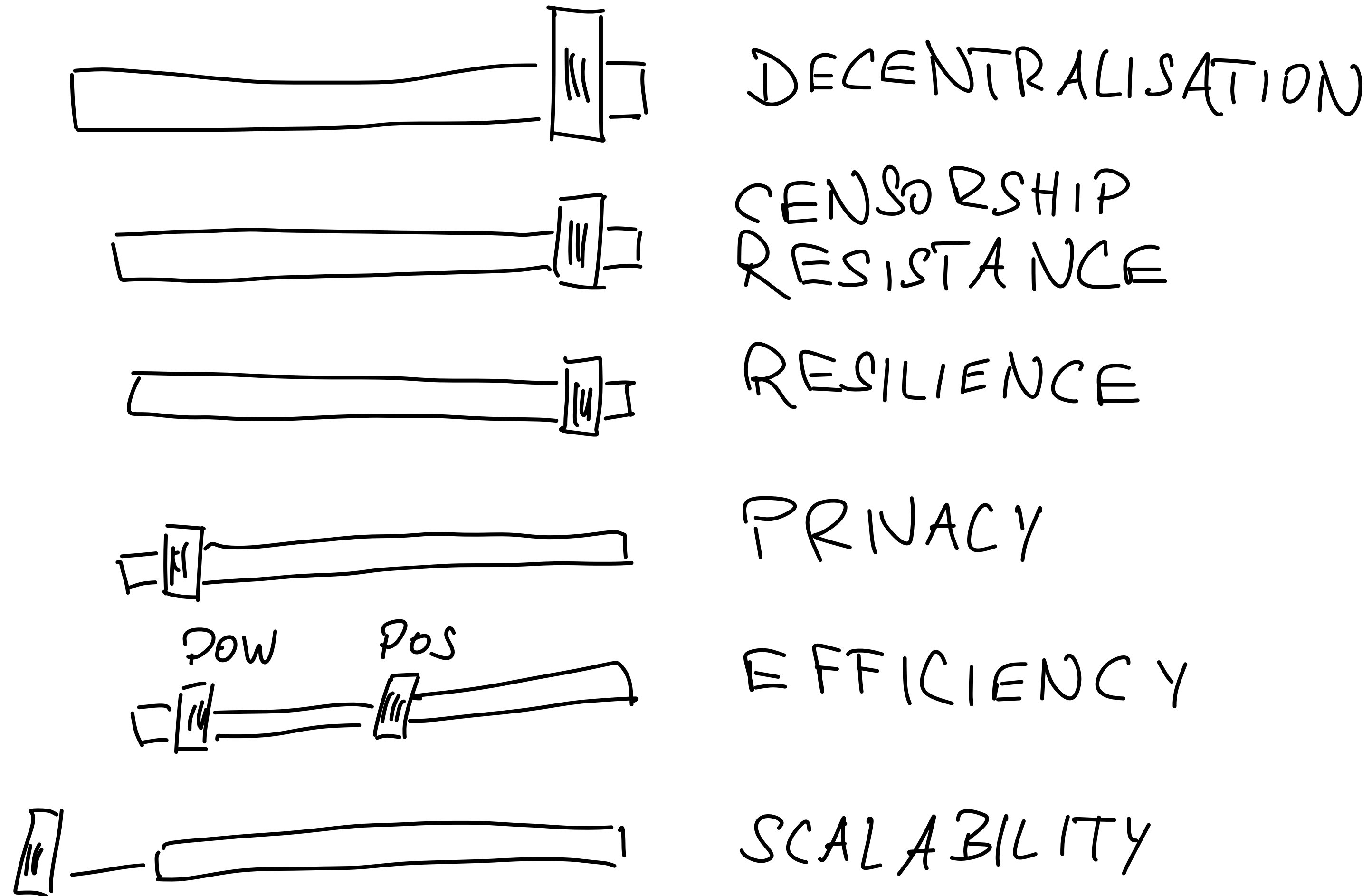


CENSORSHIP
RESISTANCE

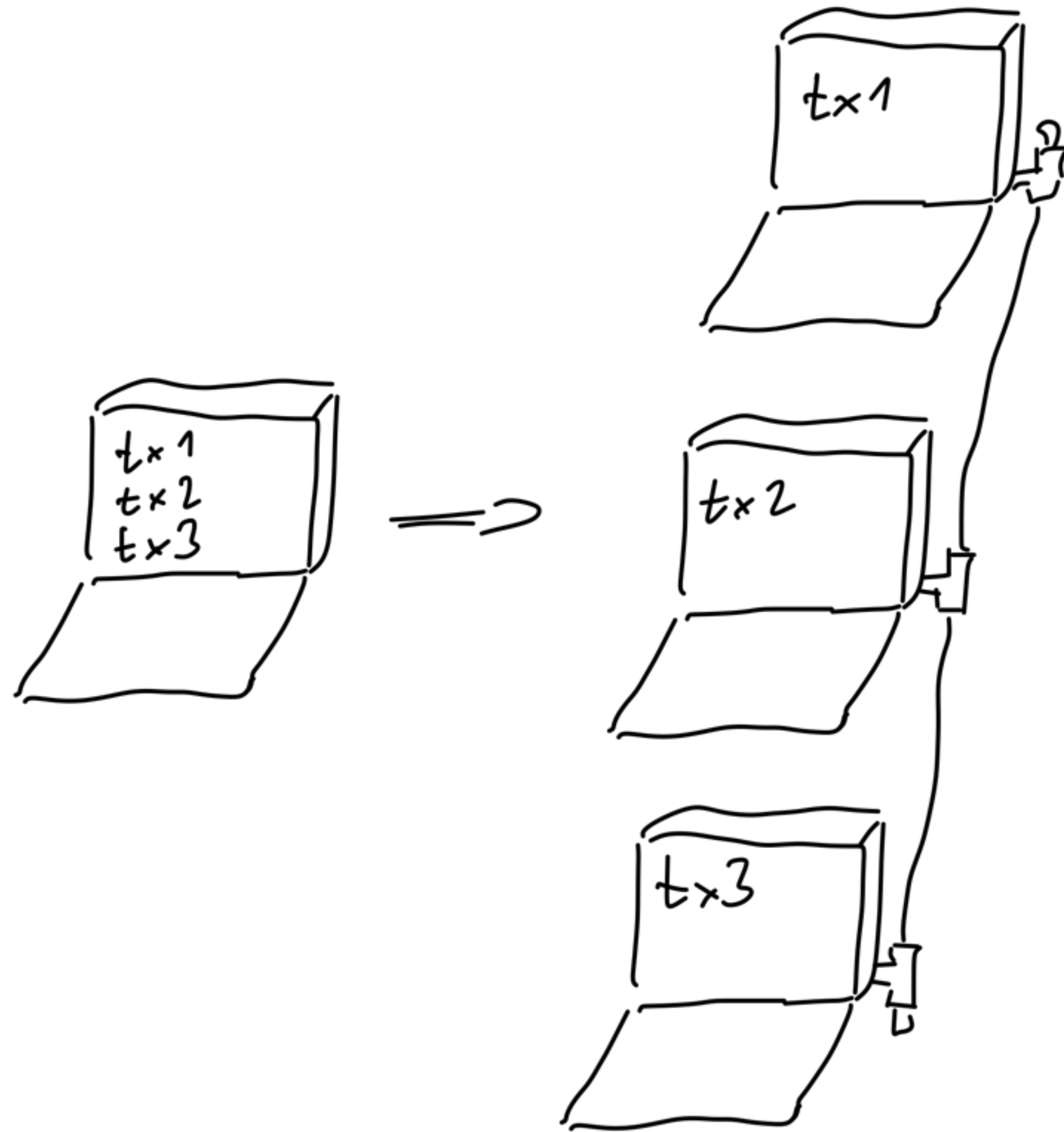


RESILIENCE

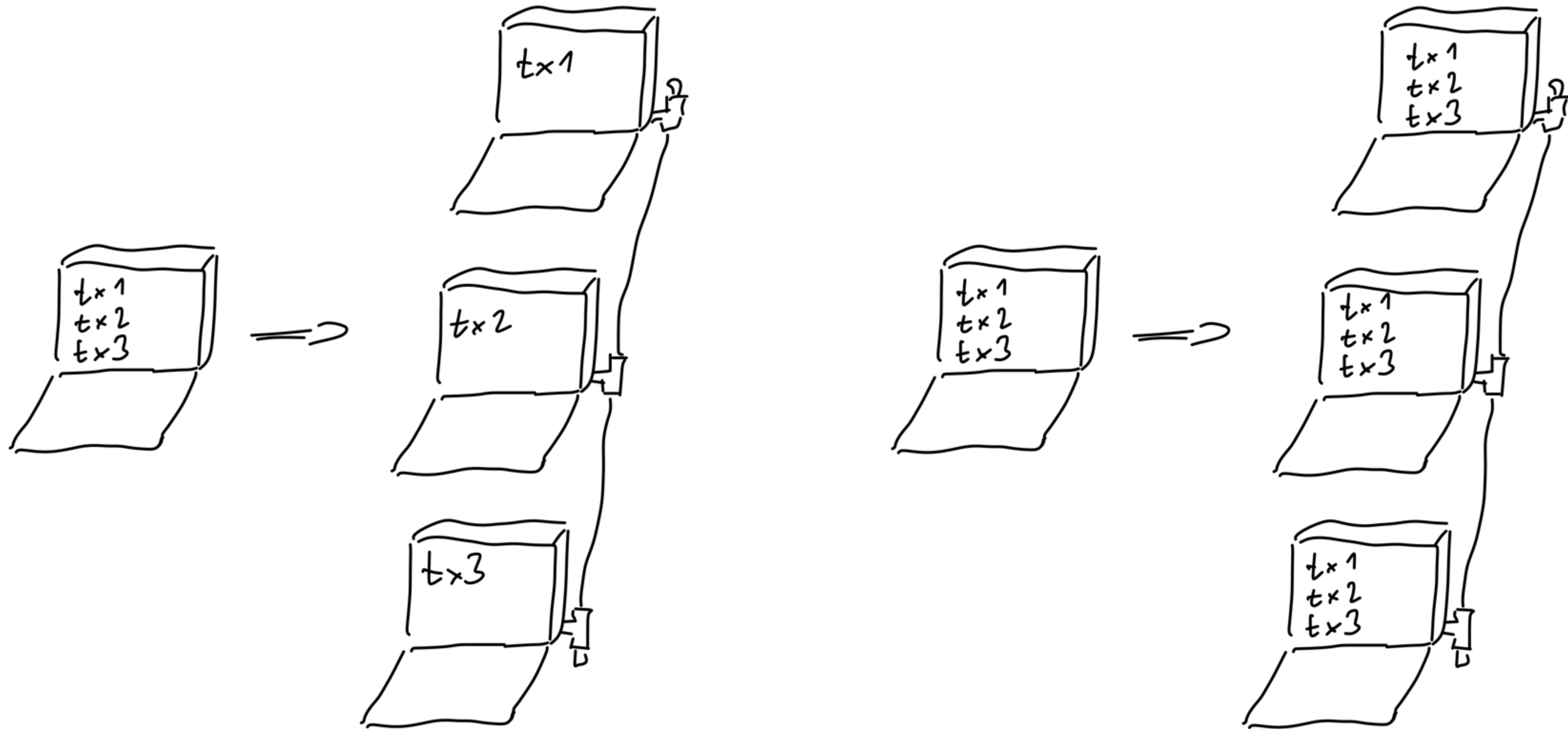
Blockchains



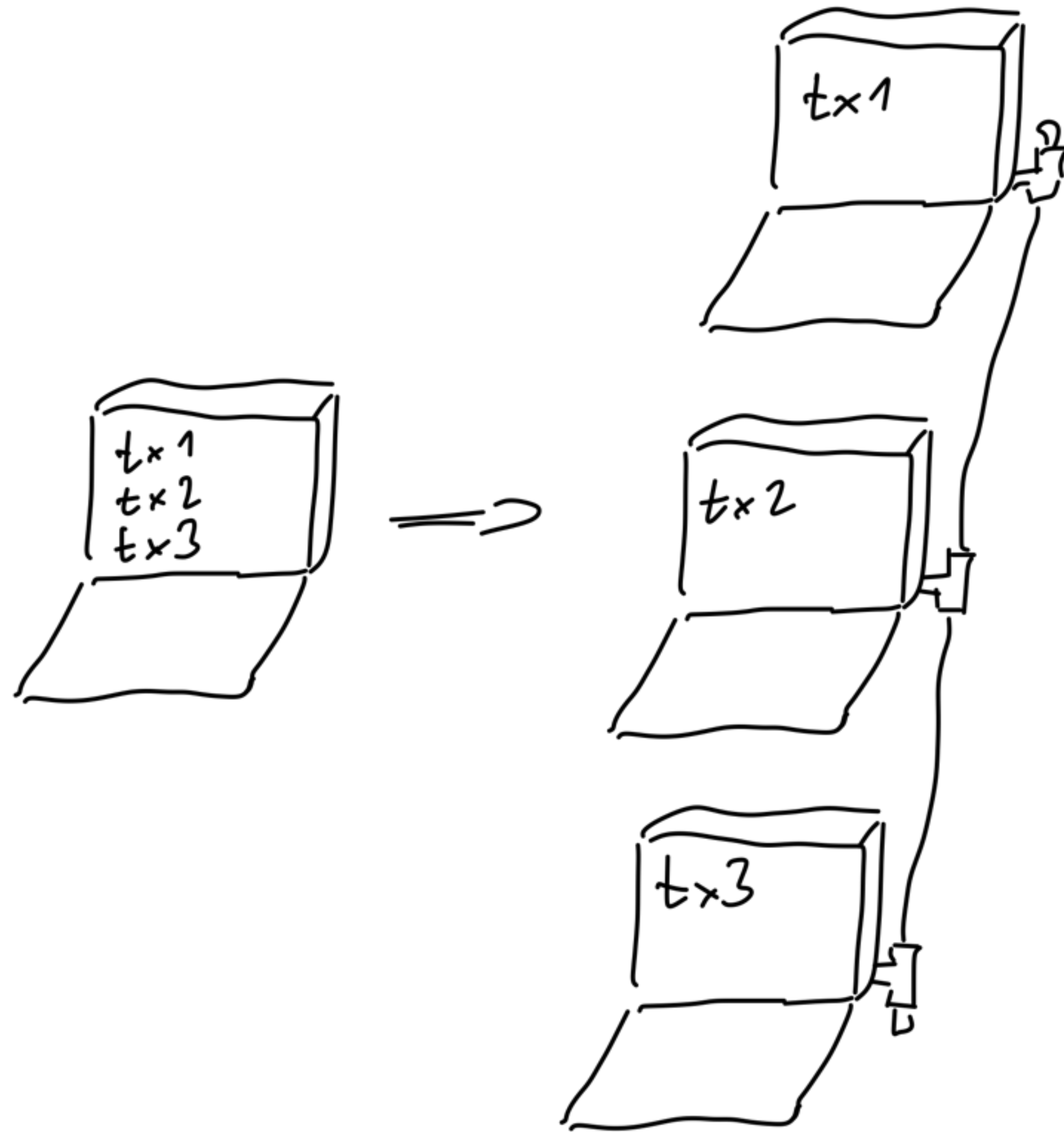
Scalability



Scalability



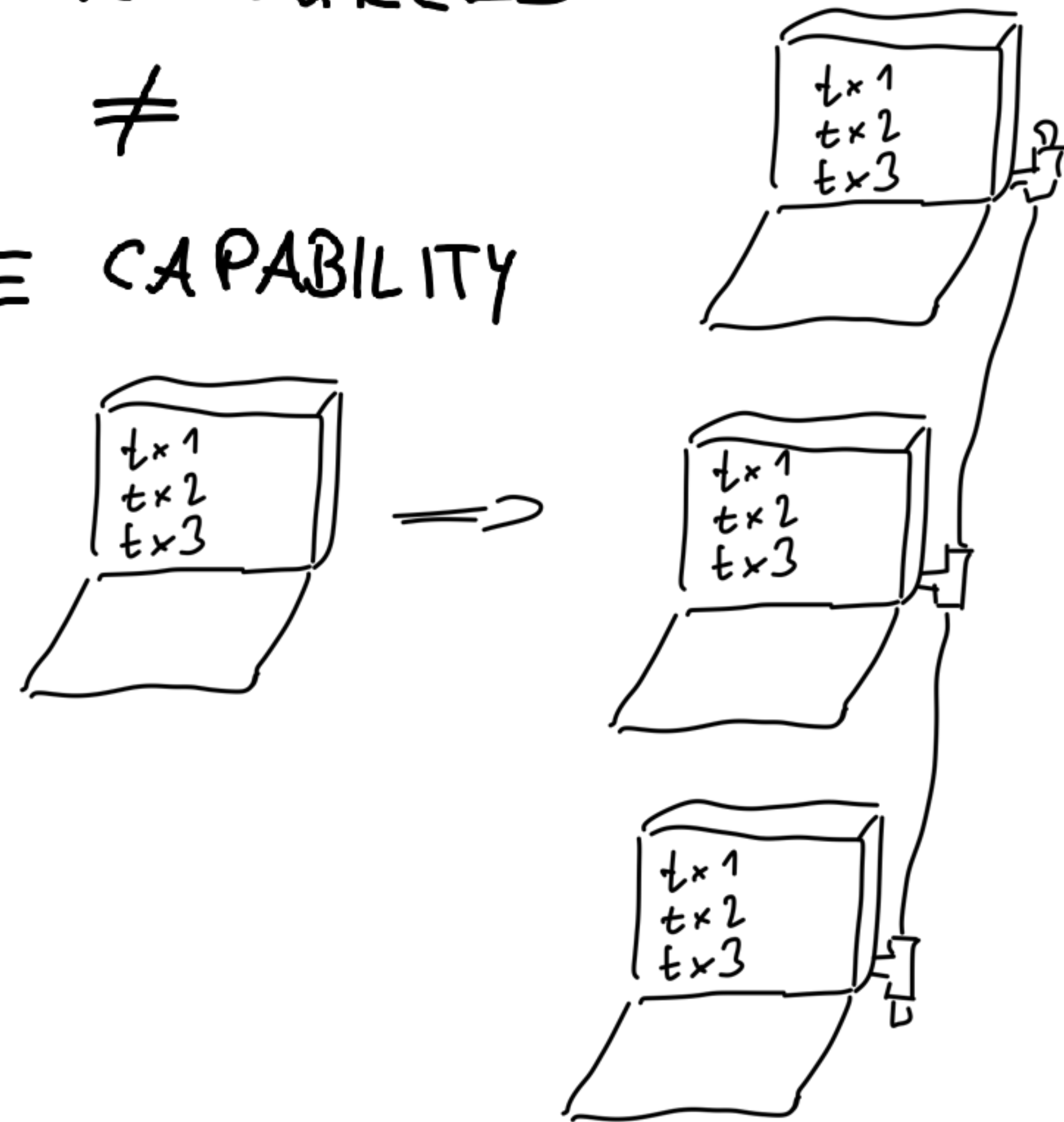
Scalability



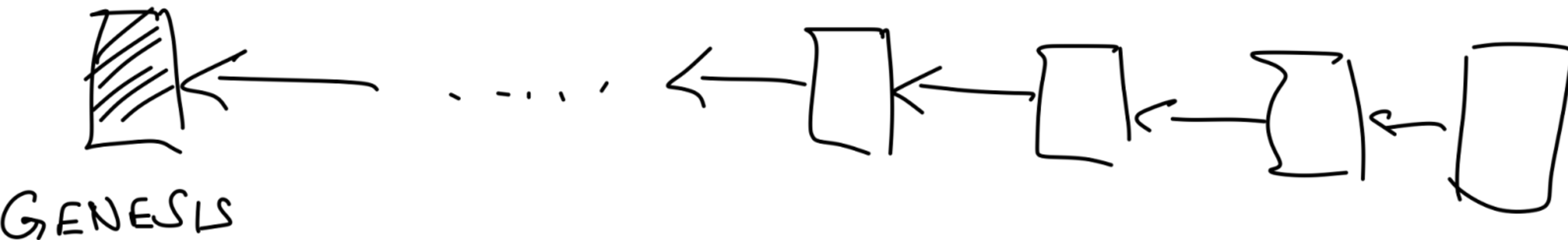
MORE RESOURCES

$\neq$

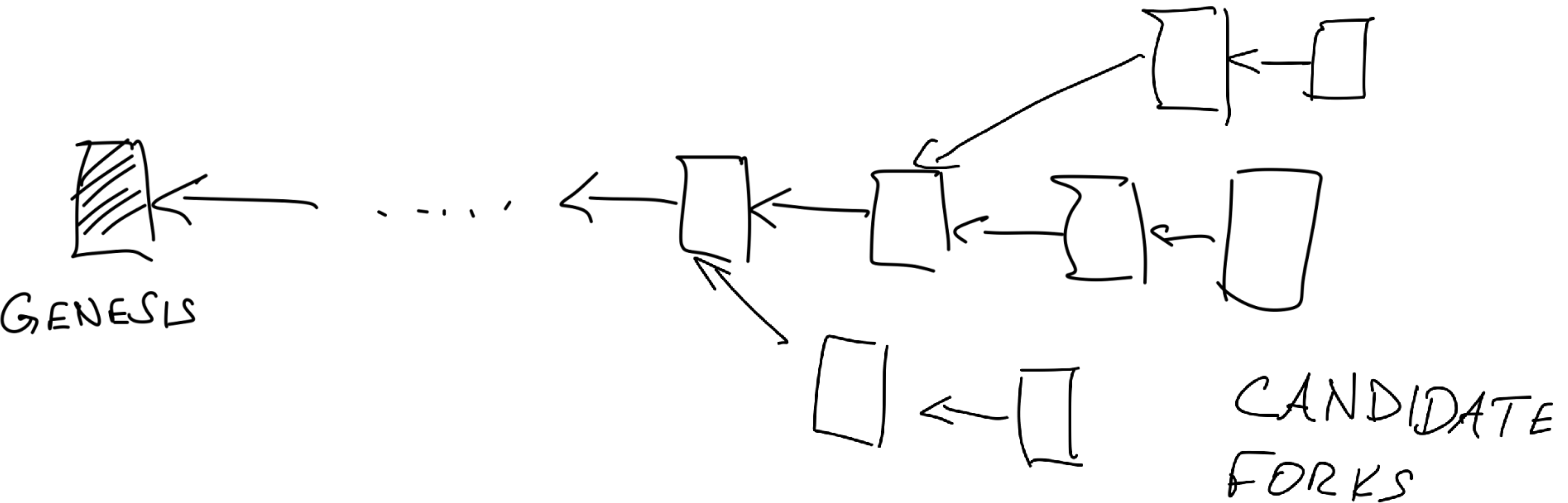
MORE CAPABILITY

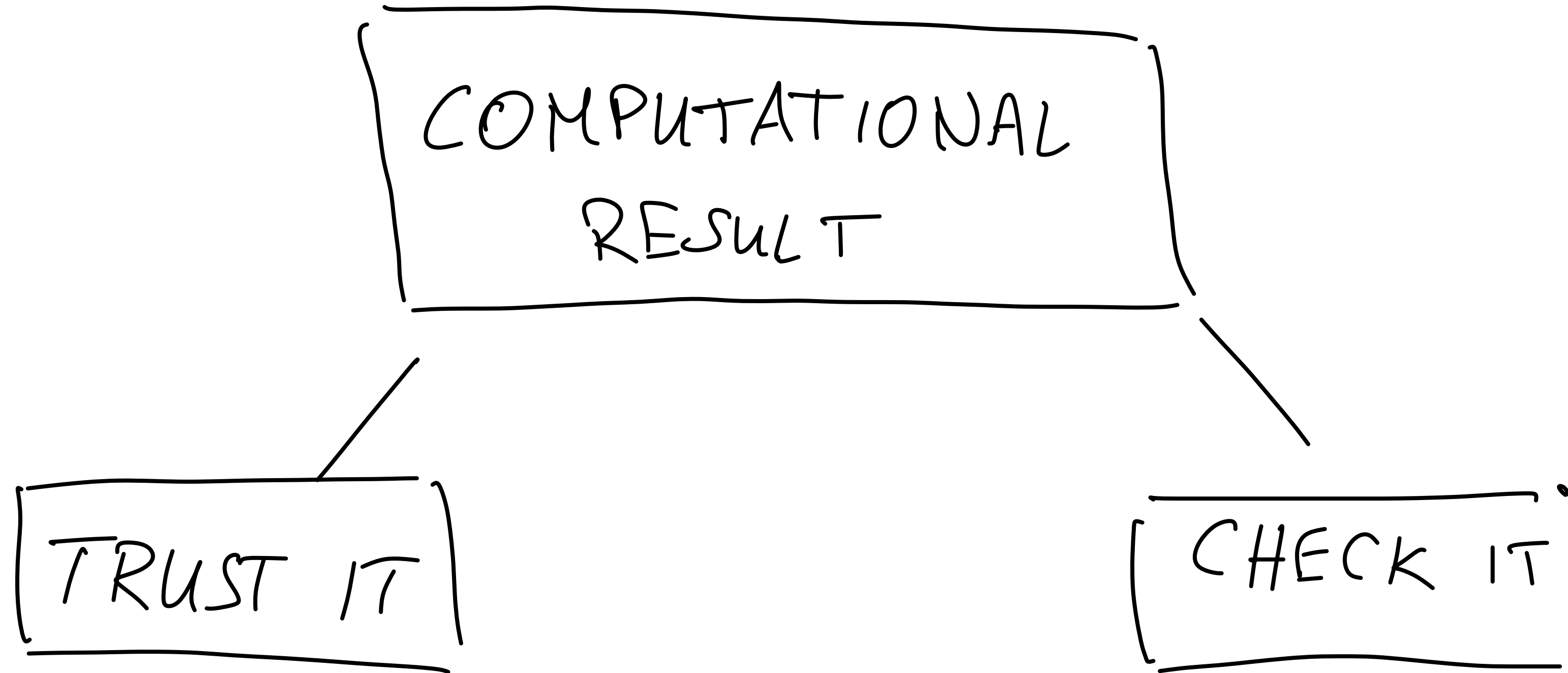


Joining the Network



Joining the Network





$\mathbb{Z}_k$

S

N

A

R

K

Z K ZERO
KNOWLEDGE

S SUCCINCT

N NON-
INTERACTIVE

A ARGUMENT
R of

K KNOWLEDGE

zk
ZERO
KNOWLEDGE

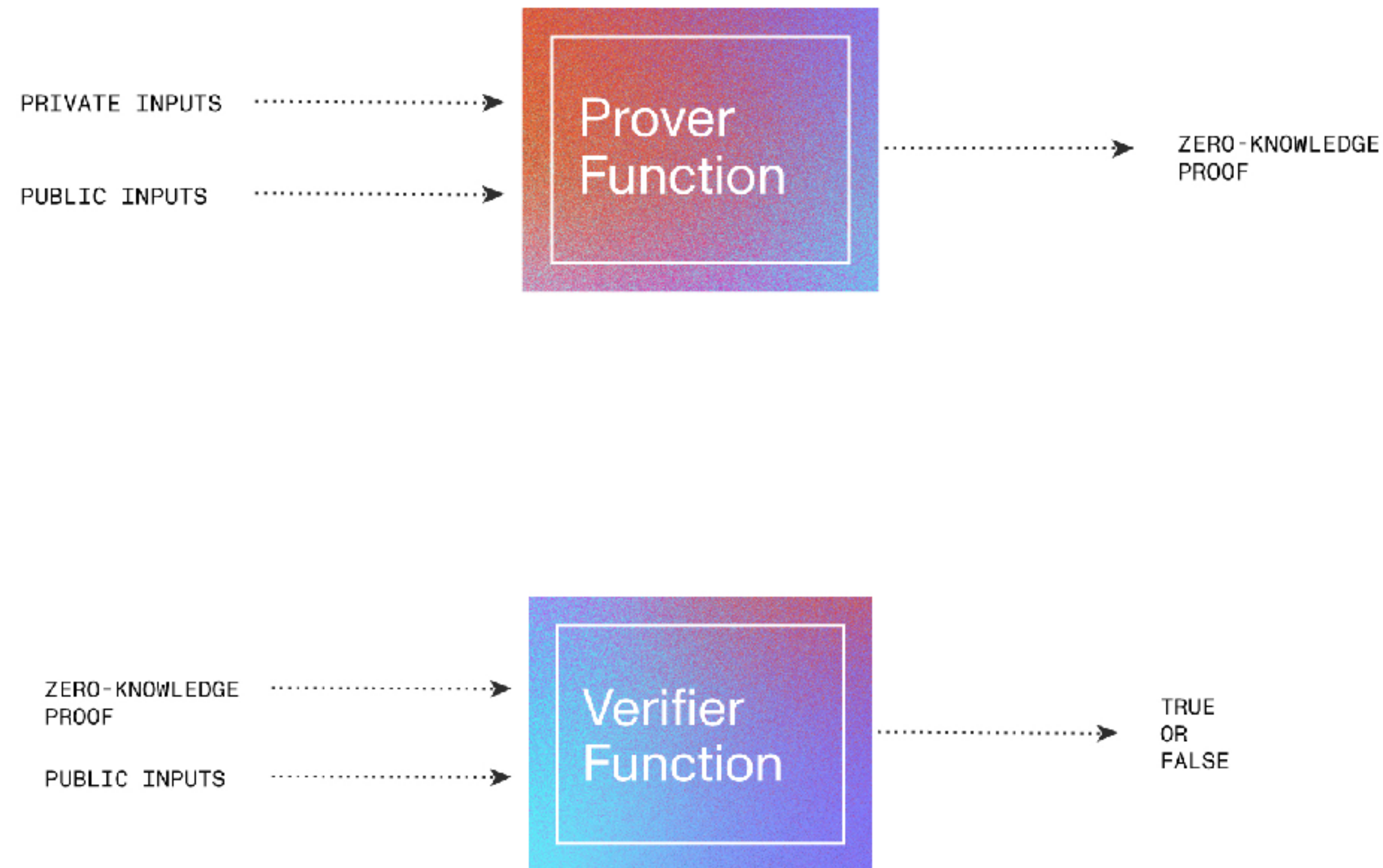
S
SUCCINCT

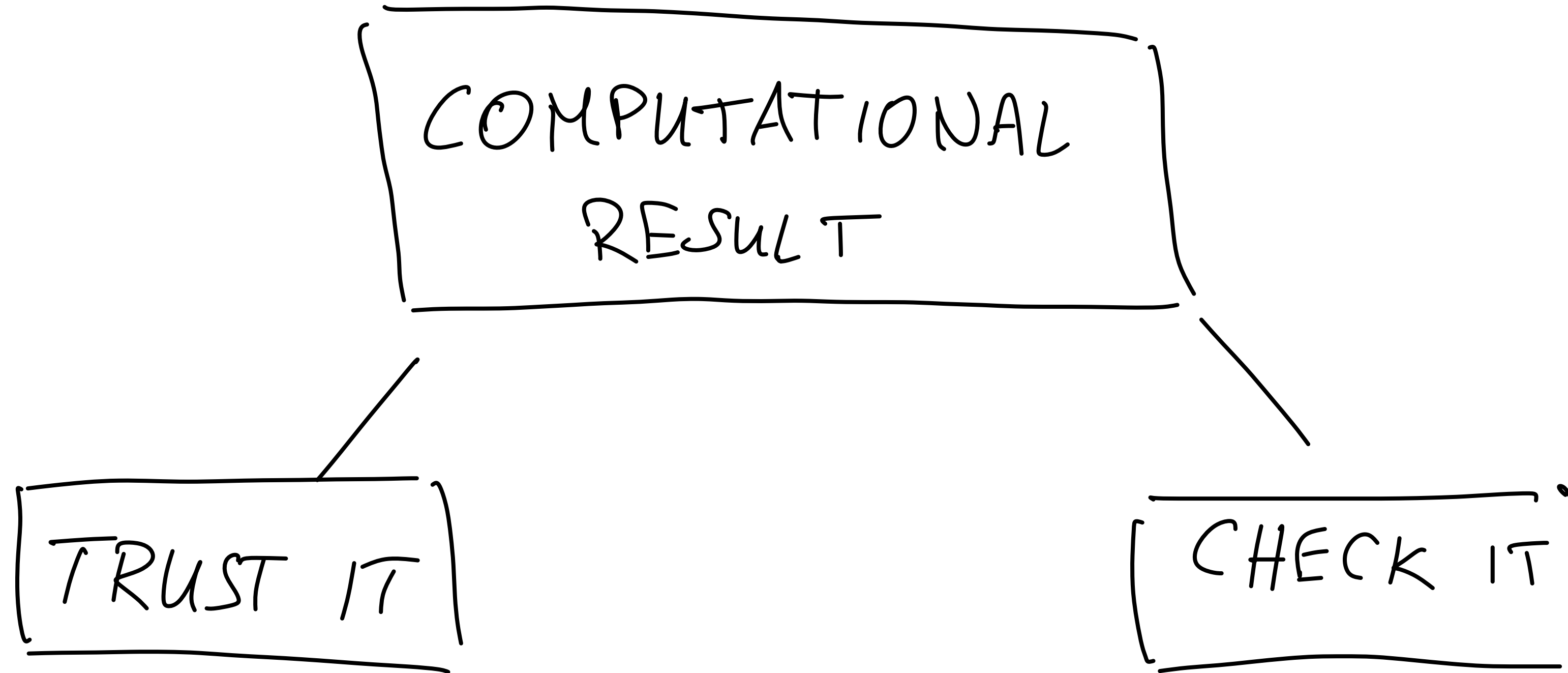
N
NON-
INTERACTIVE

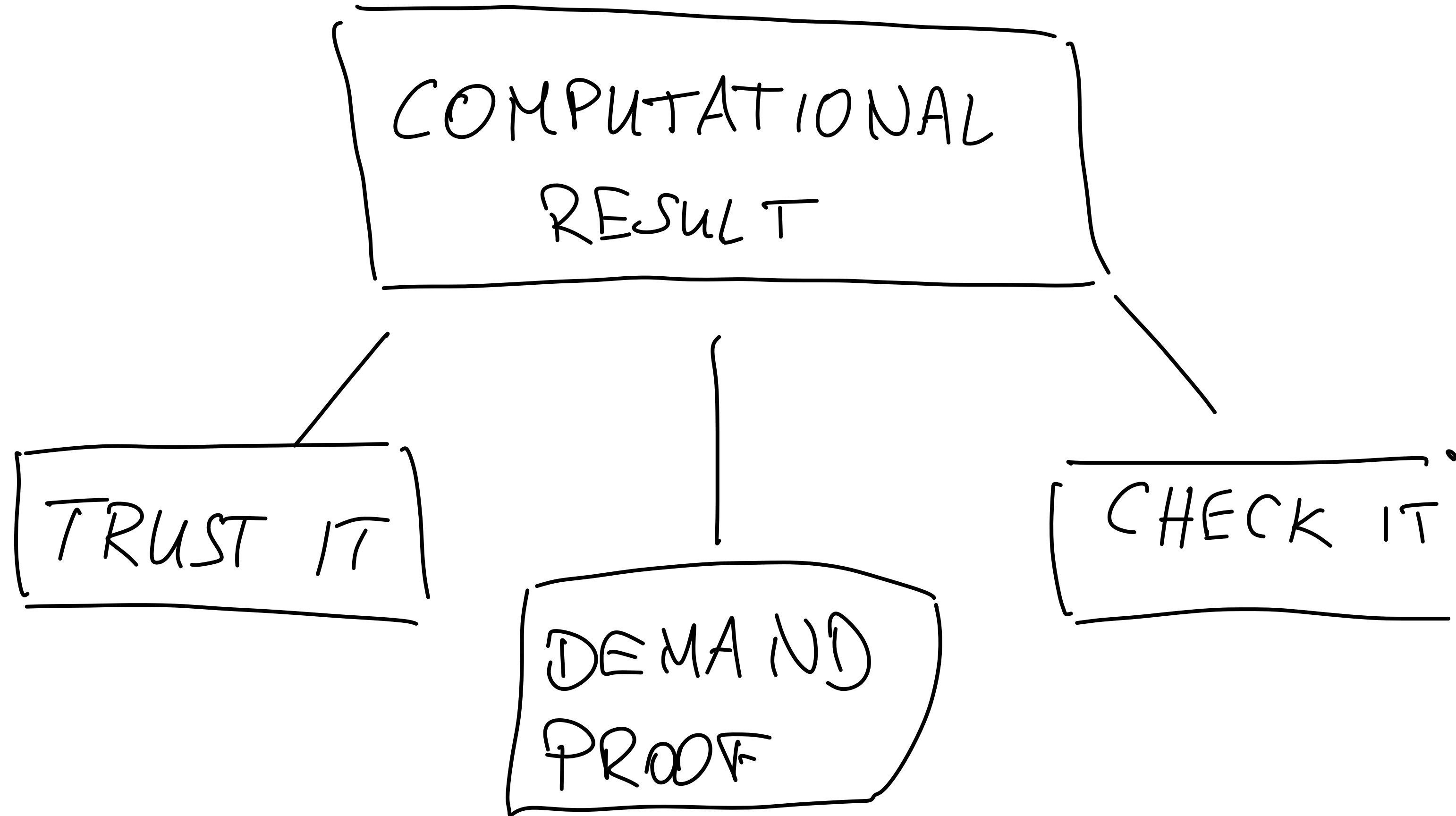
A
ARGUMENT
of

R

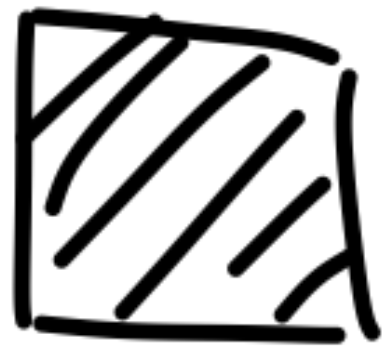
K
KNOWLEDGE





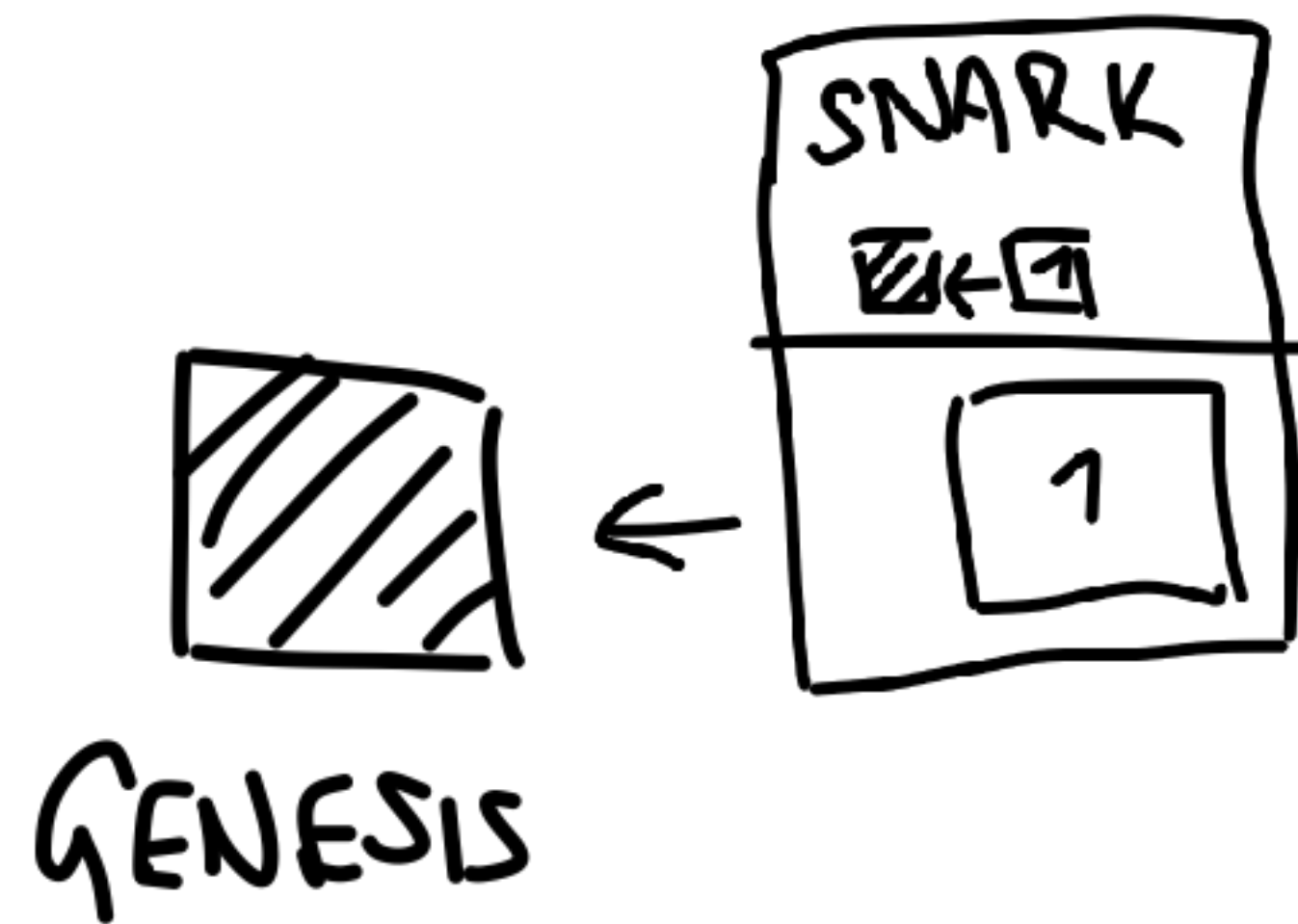


Succinct Blockchain: Ouroboros Samasika

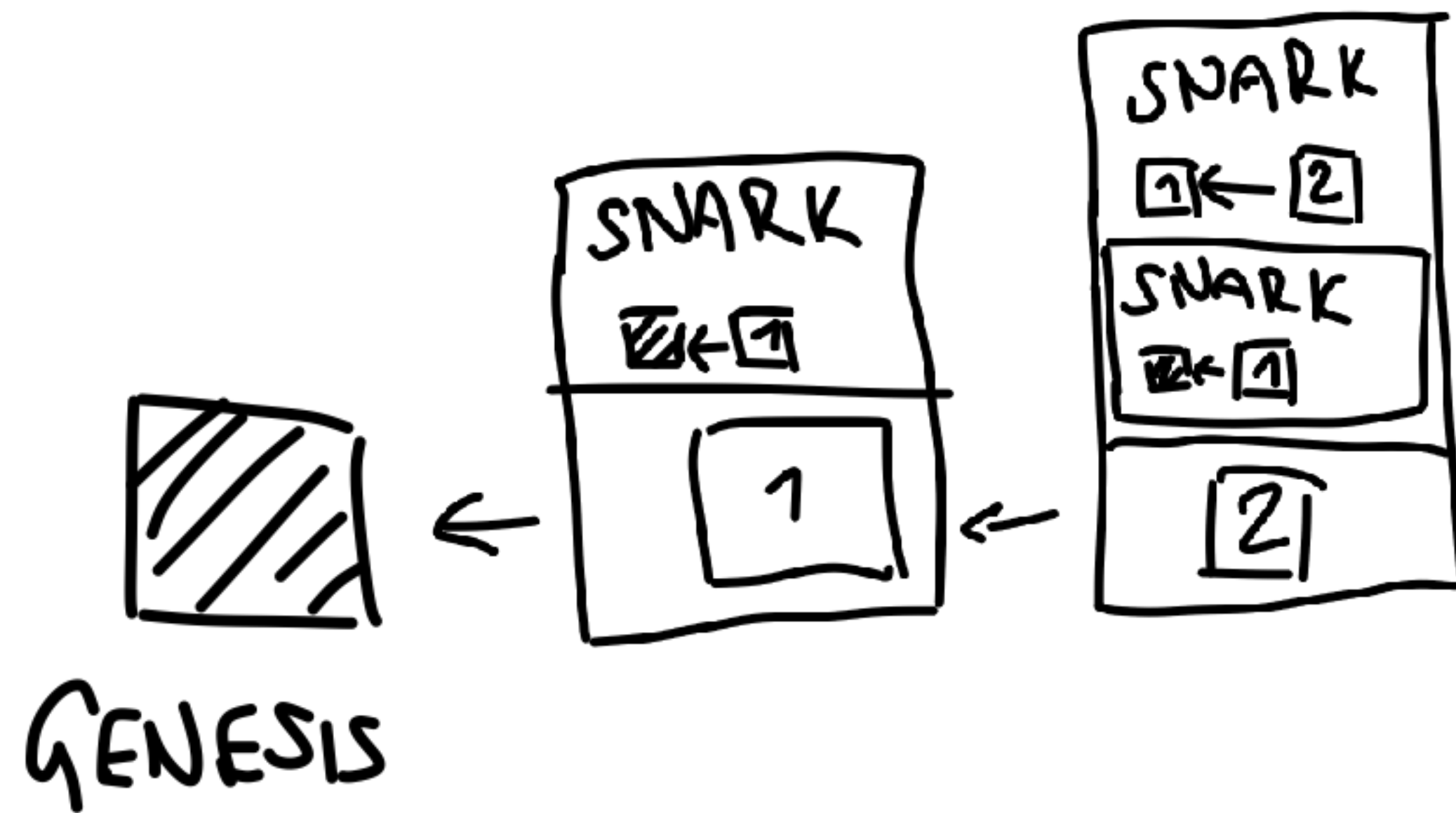


GENESIS

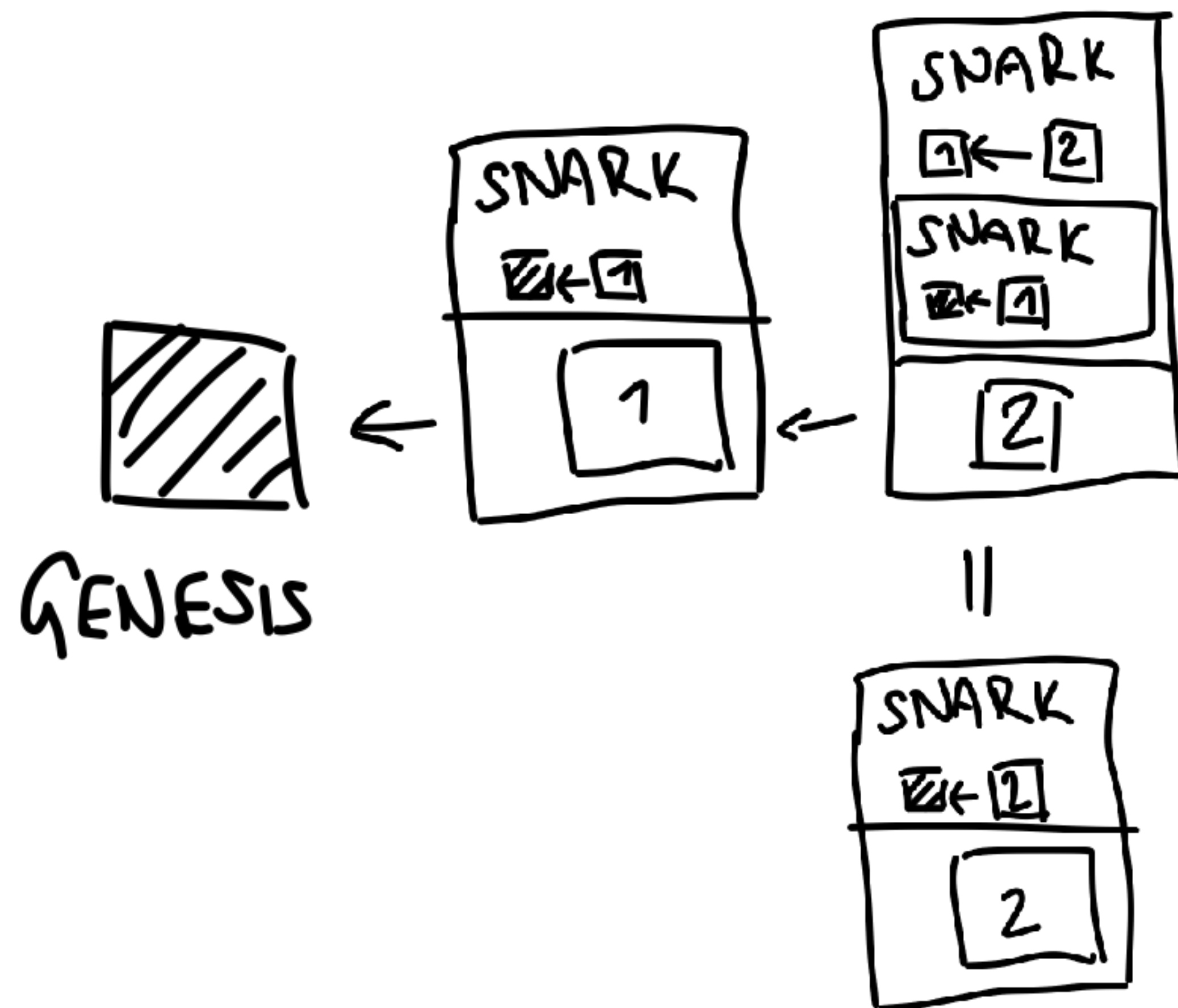
Succinct Blockchain: Ouroboros Samasika



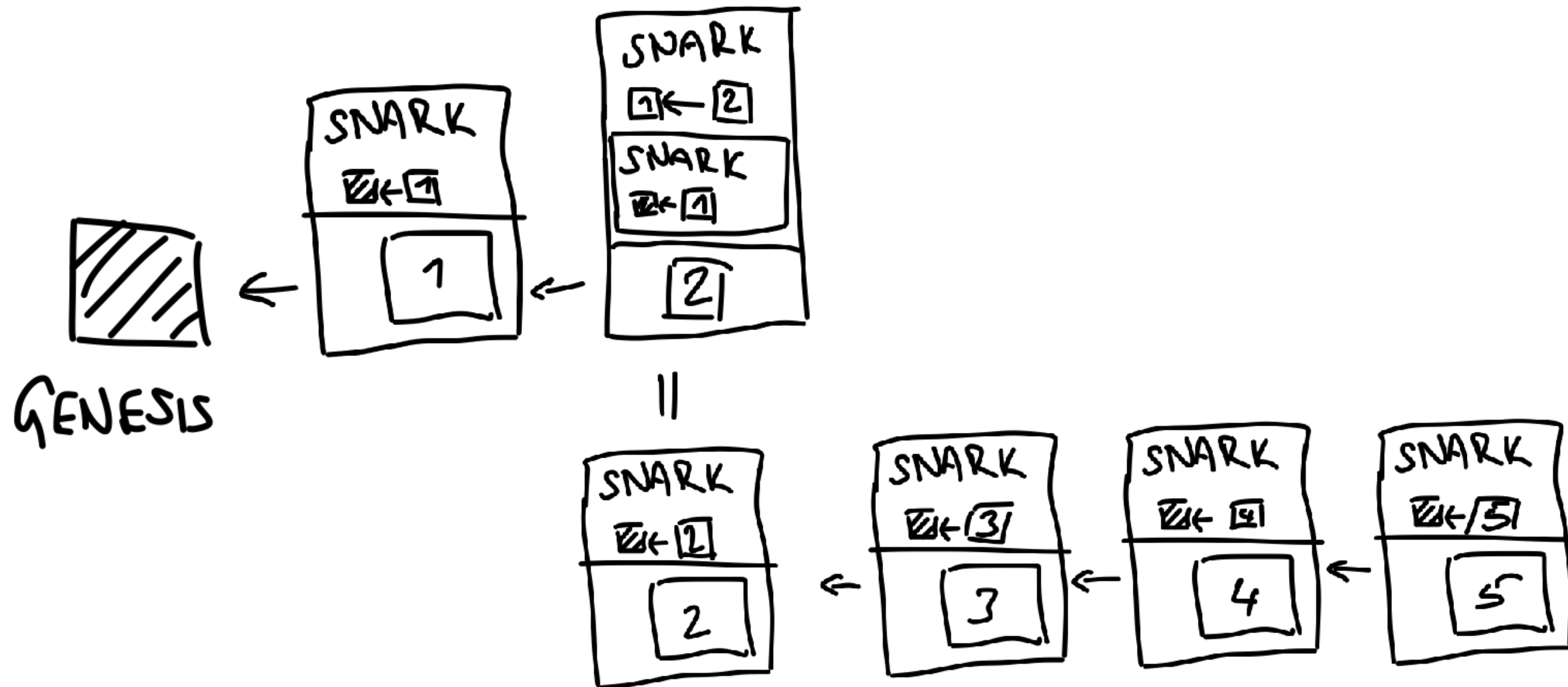
Succinct Blockchain: Ouroboros Samasika



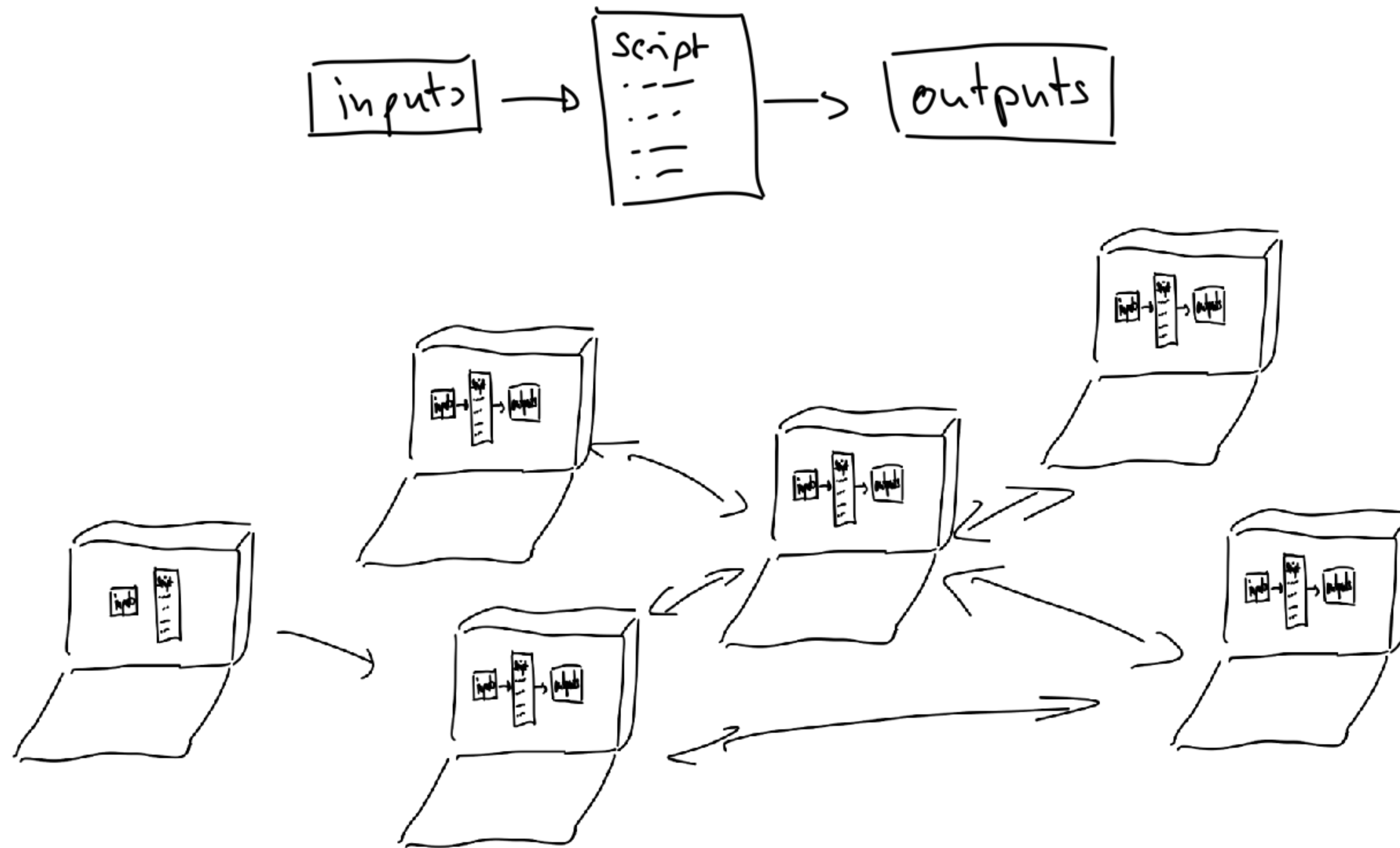
Succinct Blockchain: Ouroboros Samasika

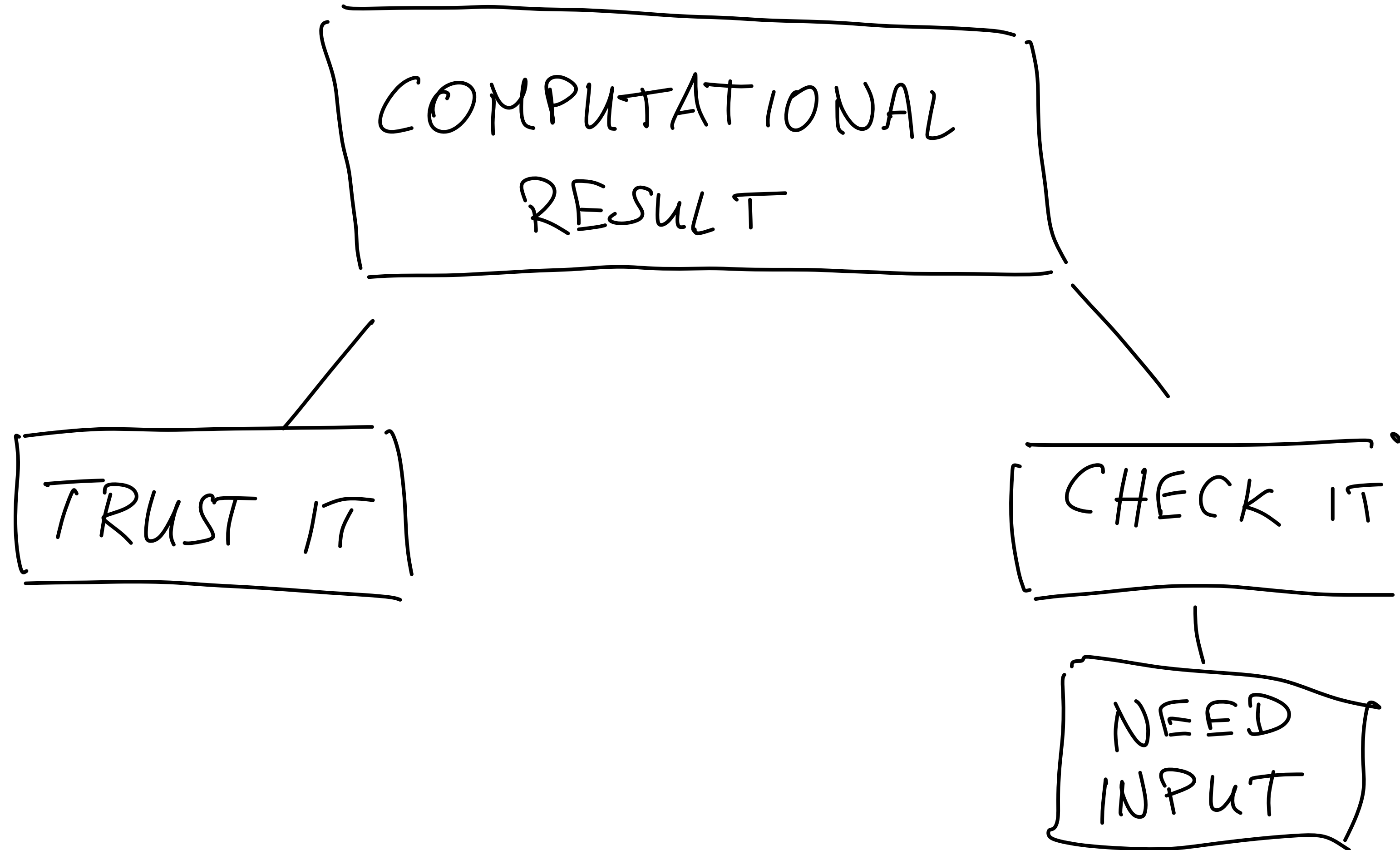


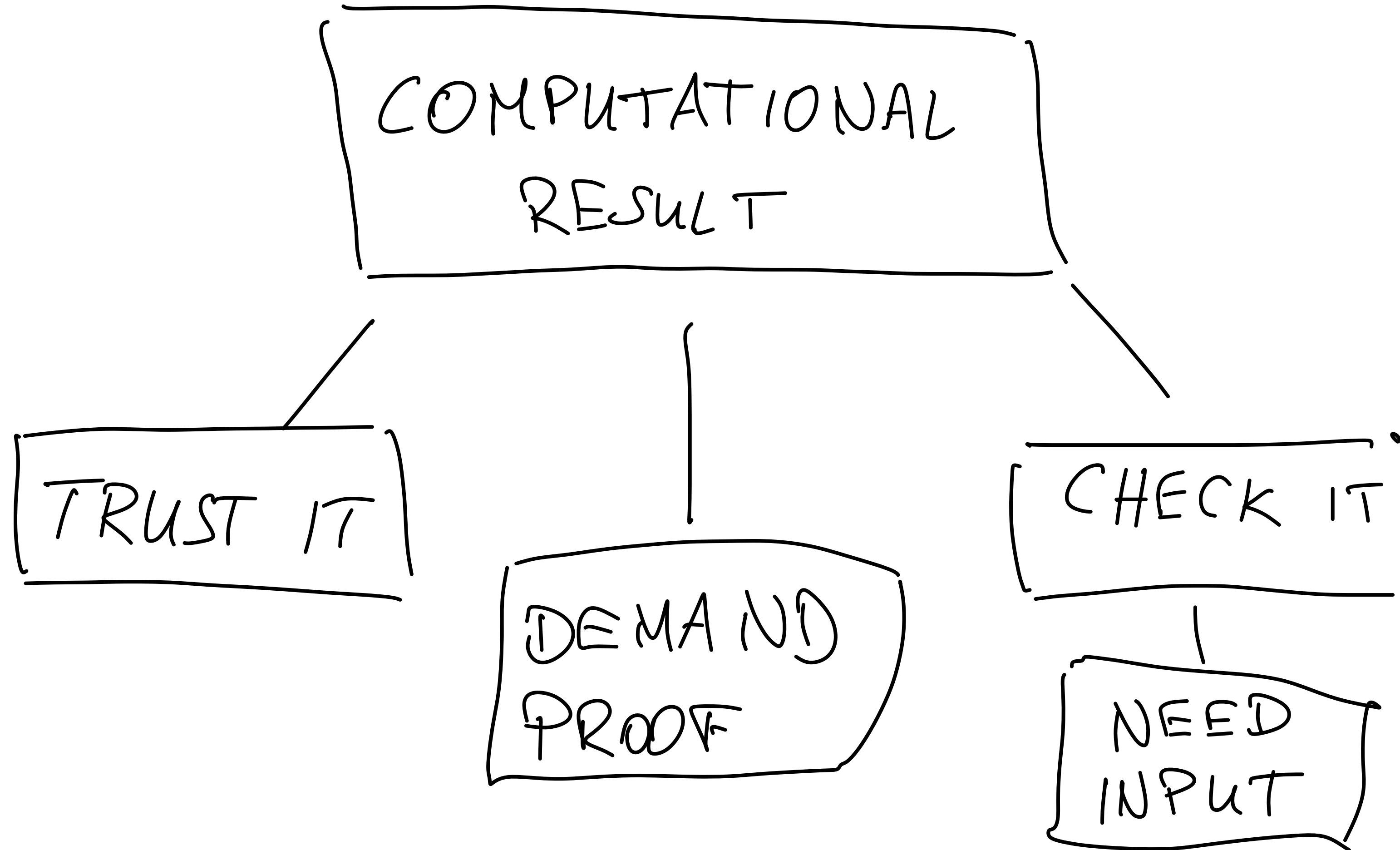
Succinct Blockchain: Ouroboros Samasika



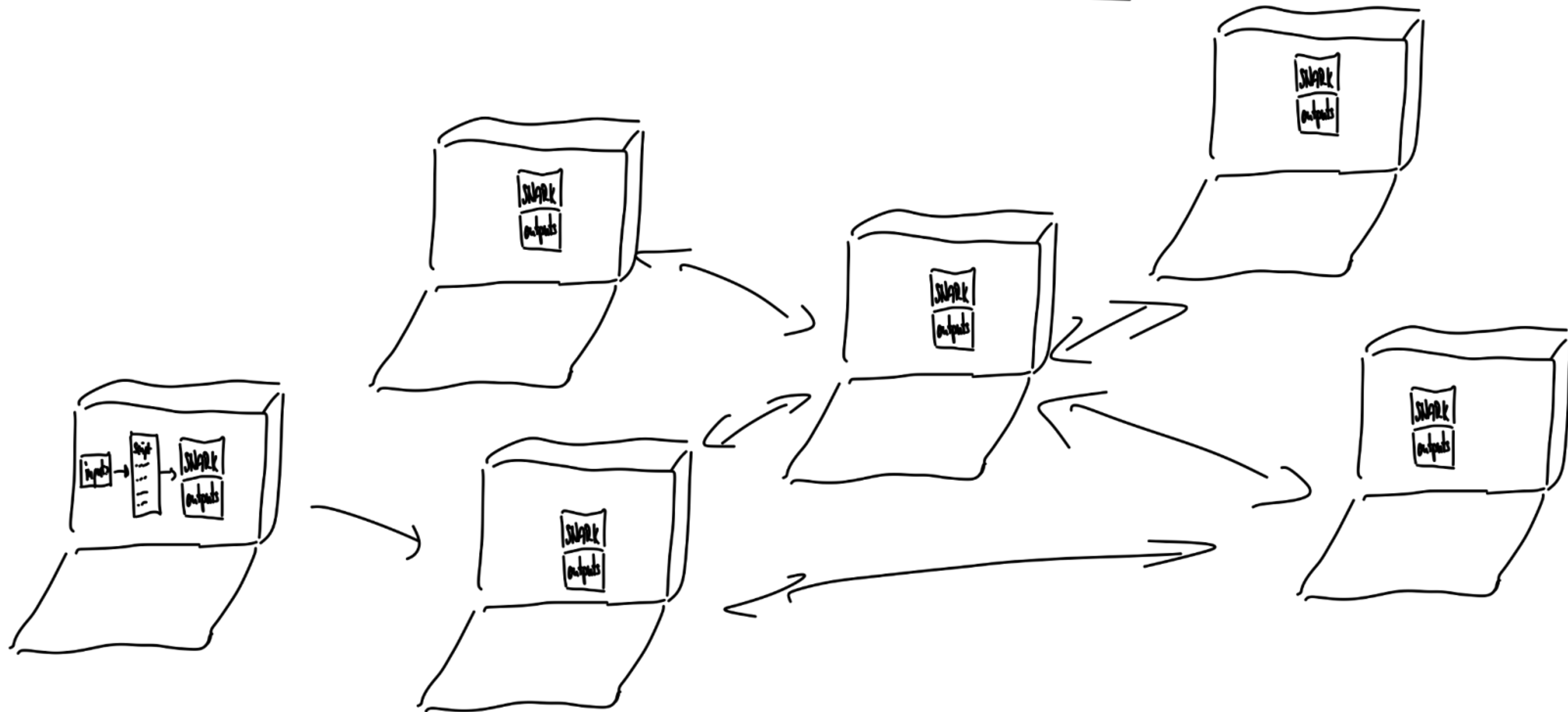
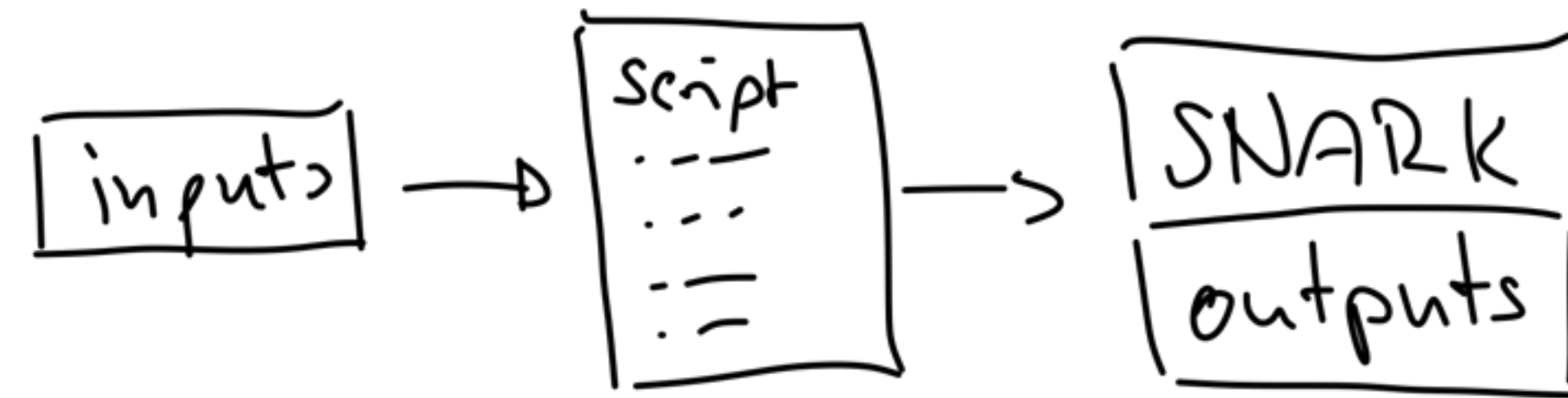
Privacy





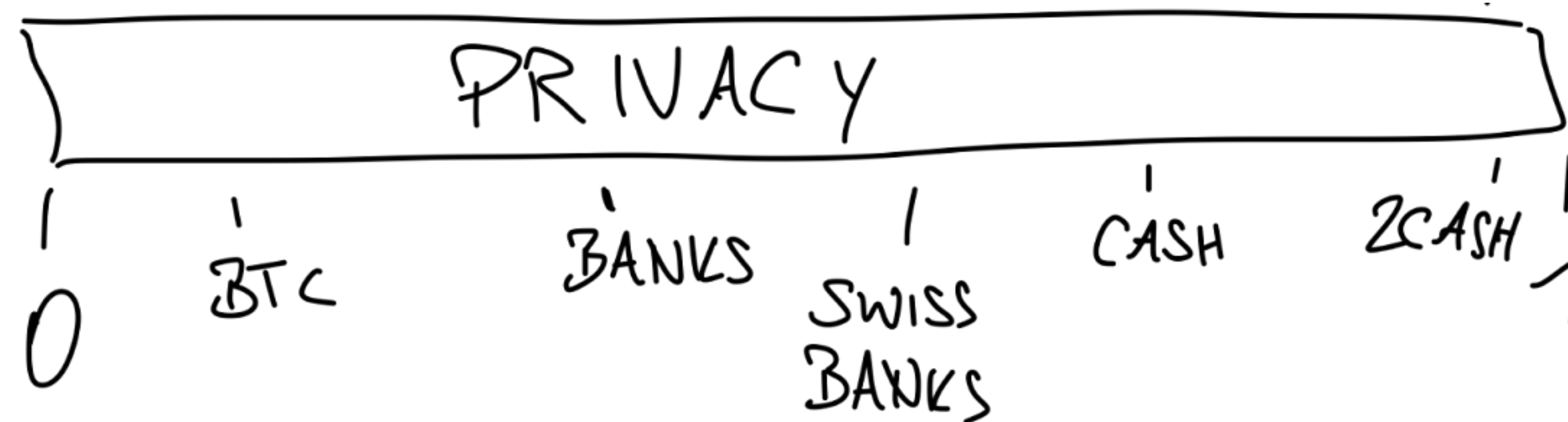


zkApps

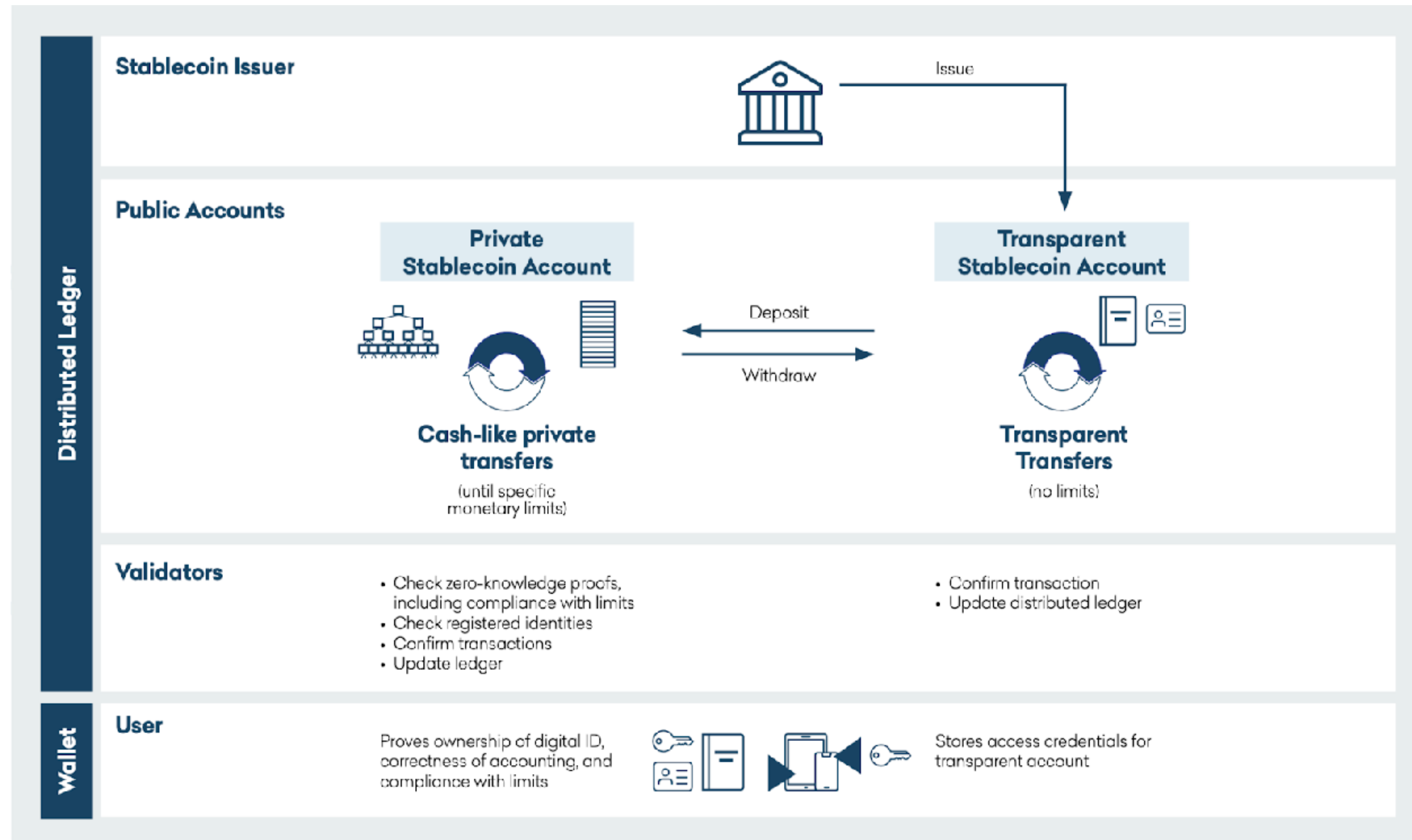


Degrees of Privacy

Subjective!



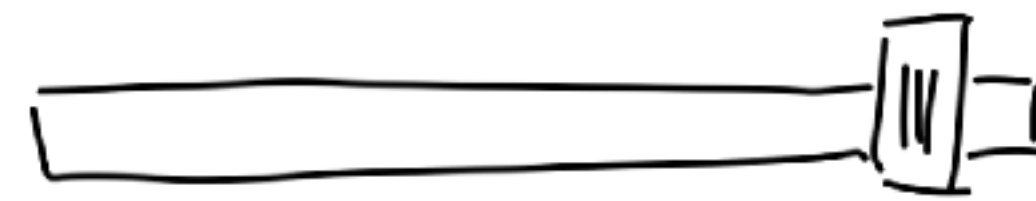
Programmable Privacy



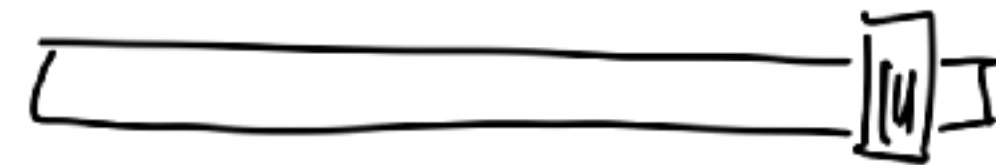
Source: based on Gross et al. (2021); applied to a decentralized stablecoin system.



DECENTRALISATION



CENSORSHIP
RESISTANCE



RESILIENCE



PRIVACY

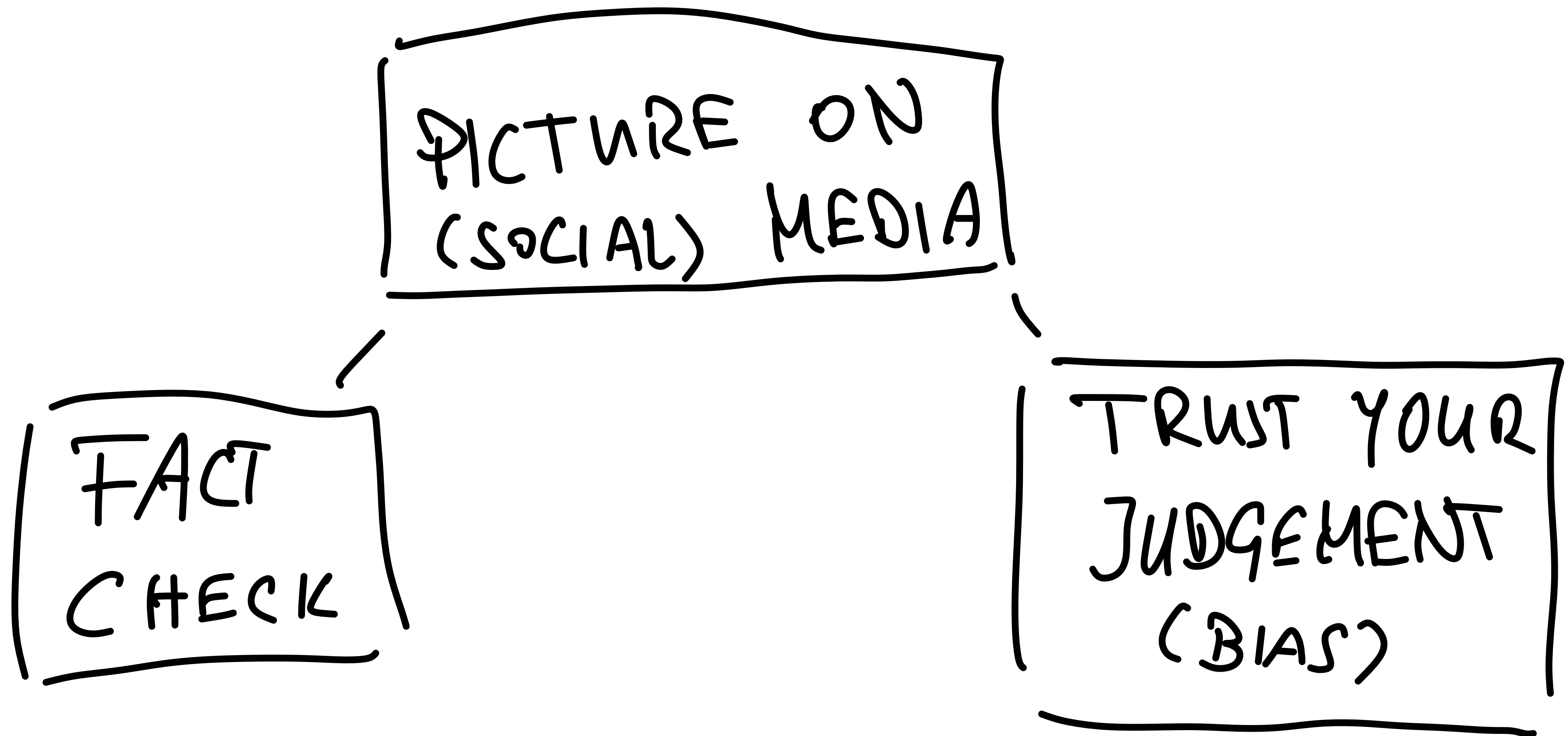


EFFICIENCY

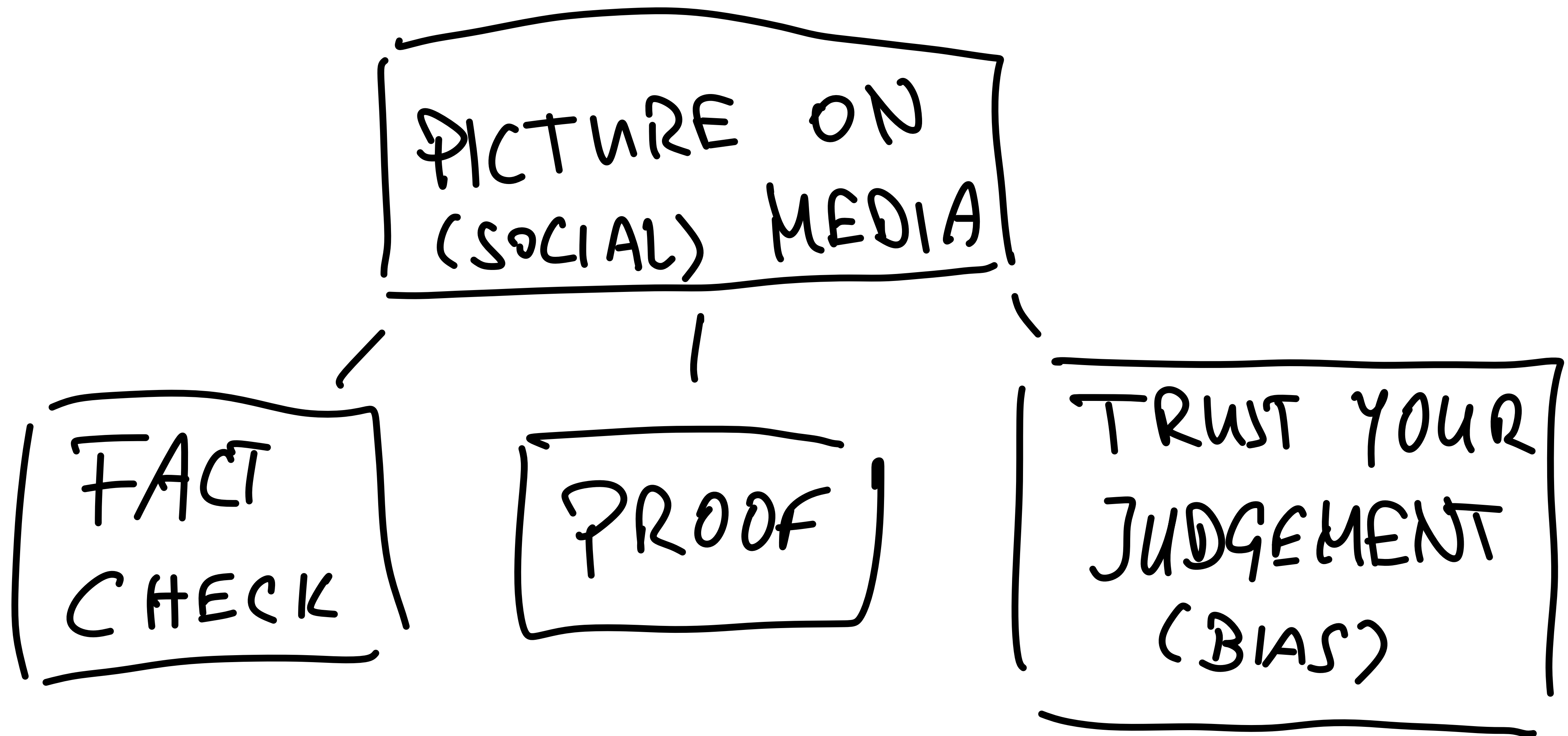


SCALABILITY

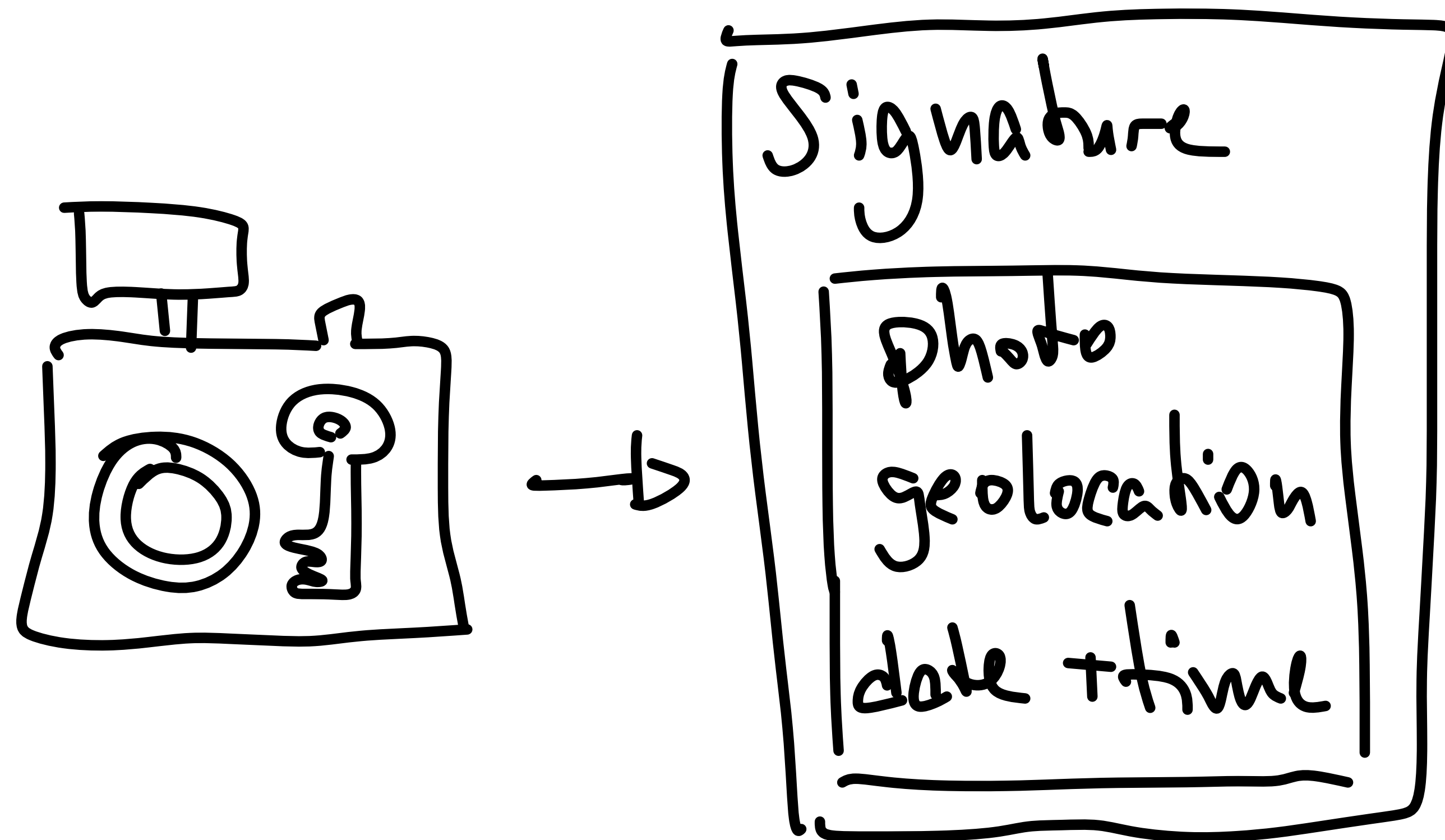
Zero Knowledge Beyond Blockchain



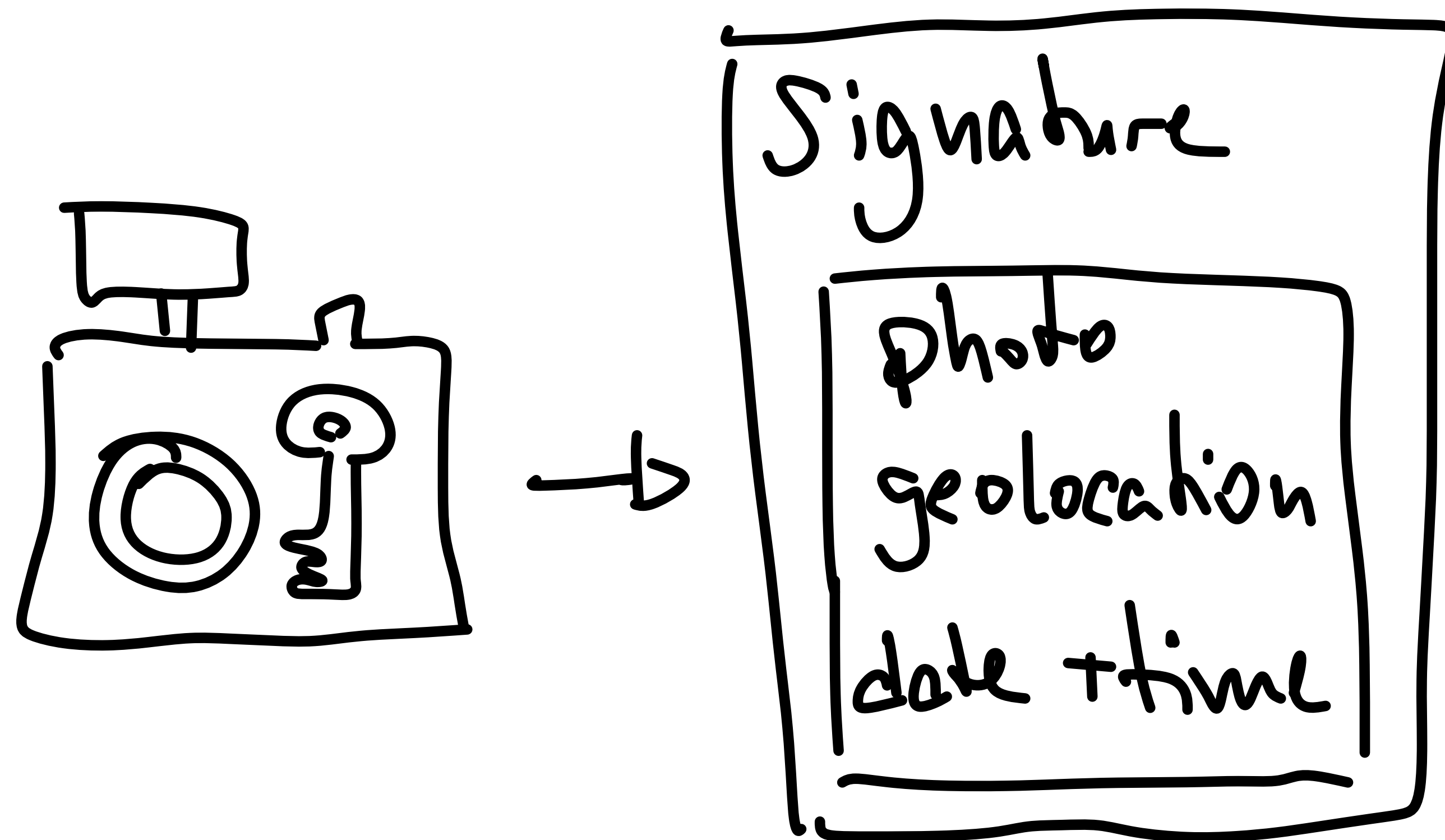
Trisha Datta, Dan Boneh: Using ZK Proofs to Fight Disinformation
<https://medium.com/@boneh/using-zk-proofs-to-fight-disinformation-17e7d57fe52f>



Trisha Datta, Dan Boneh: Using ZK Proofs to Fight Disinformation
<https://medium.com/@boneh/using-zk-proofs-to-fight-disinformation-17e7d57fe52f>



Trisha Datta, Dan Boneh: Using ZK Proofs to Fight Disinformation
<https://medium.com/@boneh/using-zk-proofs-to-fight-disinformation-17e7d57fe52f>



Problem:
publish processed
picture
→ Signature
check fail

Publisher

proof:

∃ original image & signature

∃ series of permissible
edits that result in

published picture

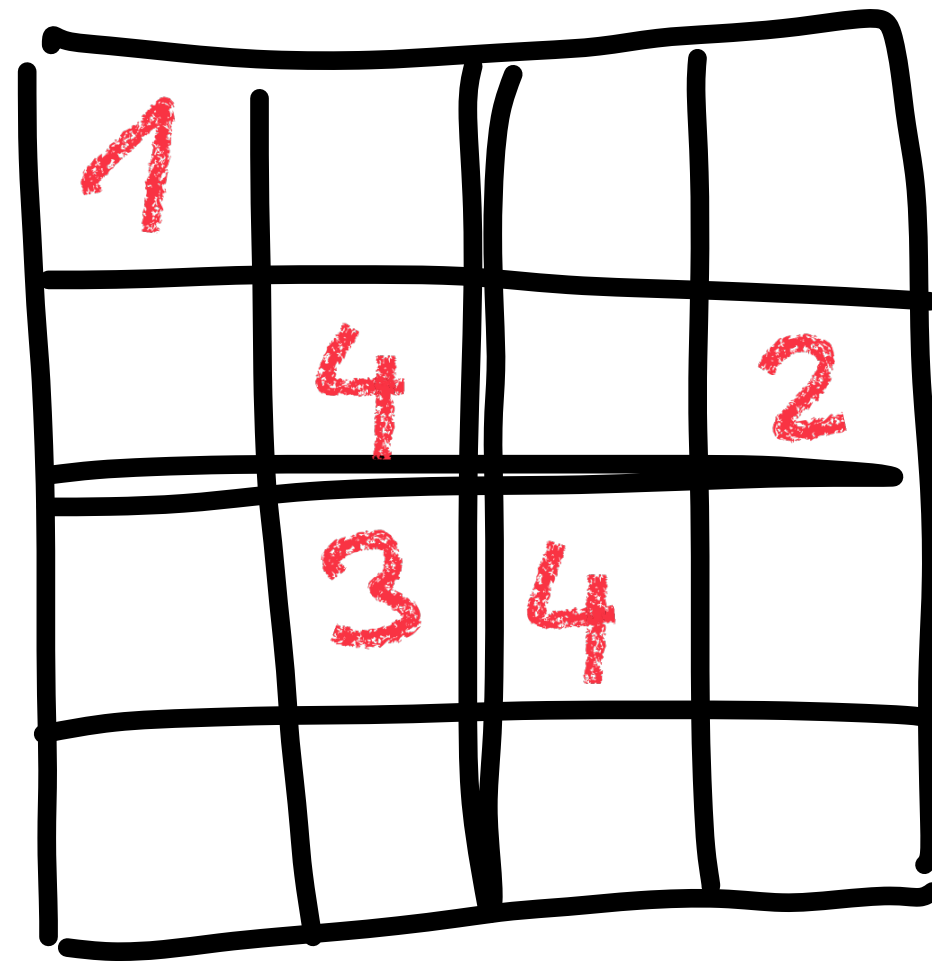
consumer (browser)

verify proof

show picture

So, How Does It Work?

Interactive Example: Sudoku



Interactive Example: Sudoku

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

Interactive Example: Sudoku

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

2	1	4	3
4	3	2	1
1	4	3	2
3	2	1	4

$1 \rightarrow 2$
 $2 \rightarrow 1$
 $3 \rightarrow 4$
 $4 \rightarrow 3$

Interactive Example: Sudoku

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

2	1	4	3
4	3	2	1
1	4	3	2
3	2	1	4

$1 \rightarrow 2$
 $2 \rightarrow 1$
 $3 \rightarrow 4$
 $4 \rightarrow 3$

Interactive Example: Sudoku

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

2	1	4	3
4	3	2	1
1	4	3	2
3	2	1	4

$1 \rightarrow 2$
 $2 \rightarrow 1$
 $3 \rightarrow 4$
 $4 \rightarrow 3$

Interactive Example: Sudoku

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

2	1	4	3
4	3	2	1
1	4	3	2
3	2	1	4

$1 \rightarrow 2$
 $2 \rightarrow 1$
 $3 \rightarrow 4$
 $4 \rightarrow 3$

Interactive Example: Sudoku

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

2	1	4	3
4	3	2	1
1	4	3	2
3	2	1	4

$1 \rightarrow 2$
 $2 \rightarrow 1$
 $3 \rightarrow 4$
 $4 \rightarrow 3$

Interactive Example: Sudoku

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

2	1	4	3
4	3	2	1
1	4	3	2
3	2	1	4

$1 \rightarrow 2$
 $2 \rightarrow 1$
 $3 \rightarrow 4$
 $4 \rightarrow 3$

iterate with new permutation

$\mathbb{Z}_k$

S

N

A

R

K

Z K ZERO
KNOWLEDGE

S SUCCINCT

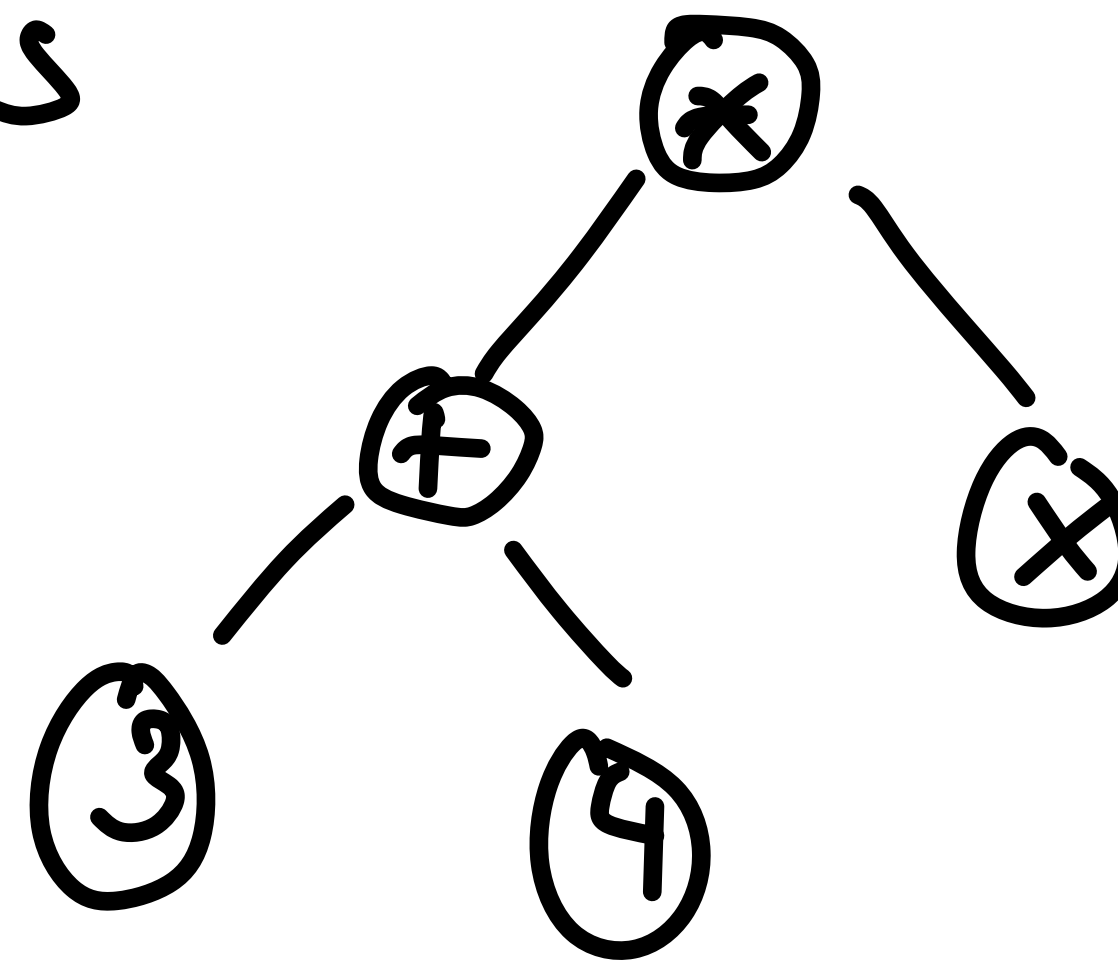
N NON-
INTERACTIVE

A ARGUMENT
R of

K KNOWLEDGE

SNARK Ingredients

- polynomials over finite fields
- elliptic curve cryptography
- commitment schemes
- arithmetic circuit



o1js: TypeScript DSL for Provable Programs

- write program in TypeScript, using o1js library
- mostly abstracts away circuits, polynomials, etc.
 - some constraints (static circuit size, control flow, ...)
- useful analogy: hardware design

```

class Sudoku extends Struct({
  value: Provable.Array(Provable.Array(Field, 9), 9),
}) {
  static from(value: number[][]): {
    return new Sudoku({ value: value.map((row) => row.map(Field)) });
  }

  hash() {
    return Poseidon.hash(this.value.flat());
  }
}

```

```

class SudokuZkApp extends SmartContract {
  @state(Field) sudokuHash = State<Field>();
  @state(Bool) isSolved = State<Bool>();

  @method submitSolution(sudokuInstance: Sudoku, solutionInstance: Sudoku) {
    let sudoku = sudokuInstance.value;
    let solution = solutionInstance.value;

    // first, we check that the passed solution is a valid sudoku
    // ...

    // next, we check that the solution extends the initial sudoku
    for (let i = 0; i < 9; i++) {
      for (let j = 0; j < 9; j++) {
        let cell = sudoku[i][j];
        let solutionCell = solution[i][j];
        // either the sudoku has nothing in it (indicated by a cell value of 0),
        // or it is equal to the solution
        Bool.or(cell.equals(0), cell.equals(solutionCell)).assertTrue(
          `solution cell (${i + 1},${j + 1}) matches the original sudoku`
        );
      }
    }

    // finally, we check that the sudoku is the one that was originally deployed
    let sudokuHash = this.sudokuHash.getAndAssertEquals();

    sudokuInstance
      .hash()
      .assertEquals(sudokuHash, 'sudoku matches the one committed on-chain');

    // all checks passed => the sudoku is solved!
    this.isSolved.set(Bool(true));
  }
}

```

Thank You!

More Information: <https://docs.minaprotocol.com/>

We're Hiring!